

DTSA 5511 · Introduction to Deep Learning · Final Project

Computer Vision in Autonomous Driving

Car Detection with a Custom Deep Learning Model
Built from Scratch



Ilya Petrusenko

https://github.com/ilyapt/DTSA5011_Final_Project

Abstract

This project explores object detection on the KITTI dataset of images from cameras of autonomous cars. The goal was not to use a ready-made model and achieve the best prediction, but to understand how object detection works and to build a detection model from scratch using only standard Keras and TensorFlow tools. The model follows a highly simplified YOLO concept: the image is divided into a 39×12 grid of cells, and for each cell a convolutional network predicts the probability of a bounding box center together with the position and size of the box; it is trained with a custom loss function and post-processed with Non-Maximum Suppression. Hyperparameters were selected with Random Search, and the detection threshold was chosen using F1 Score and Matthews Correlation Coefficient. Despite being significantly simplified, the model was able to detect over 75% of cars in the validation set, and its results are compared with a COCO pre-trained YOLOv8 model fine-tuned on the same data.

Contents

1	Description of the problem	2
2	Brief description of the data	3
3	Exploratory Data Analysis (EDA)	5
4	Focus on a single class (Car)	7
5	Single-Class Exploratory Data Analysis (Car)	8
6	Plan of model architecture and learning based on EDA	9
7	Data preprocessing and transformation	10
8	Model implementation	13
8.1	Loss function	14
9	Hyperparameters tuning	15
10	Training and evaluating	16
10.1	What the model predicts	18
10.2	Choosing the threshold	20
10.3	Bounding boxes and Non-Maximum Suppression	21
11	Alternative model: pre-trained YOLO	24
12	Model comparison	29
13	Conclusion	30
	References	31
A	Supplementary code	32

1 Description of the problem

For my final project, I chose the famous KITTI dataset [1], namely the part of it that includes photos from cameras of autonomous cars.

We covered binary and multi-class classification in our courses, but when I saw this dataset, I became curious about how to do this, because:

- an image can contain many objects of the same class, and neither binary nor multi-class classification is suitable for this;
- we need to not only predict with certainty the presence of an object in the image, but also find its exact position and boundaries.

When I first thought about this, I realized I had absolutely no idea how to do it. And this is a very common task, because besides autonomous machines, a similar computer vision approach can be applied, for example, in robotics, agriculture, medicine, welding, or even simply finding faces in photographs (Figure 1).

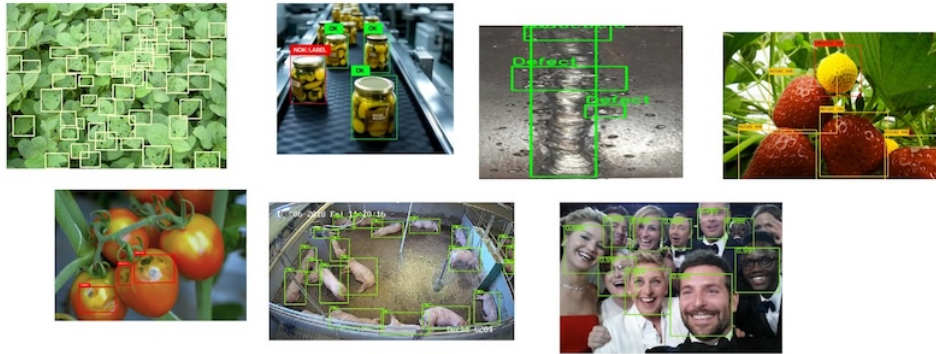


Figure 1. Examples of object detection applications outside of autonomous driving.

My final project goal **wasn't to use a ready-made model** and achieve the best prediction, but to **understand how to do it** using only **standard Keras and TensorFlow** tools, even if my model's predictions were worse than those of ready-made models. It's important to understand the underlying principles, the appropriate model architecture for this task, its outputs, activation functions, loss calculation, and so on.

My model should:

- find objects in an image and return a probability for each one;
- find the boundaries of each detected object and return their coordinates.

After I create my model, train it, and get satisfactory results, I'll compare its performance with one of the existing ready-made object detection models, like YOLO [2, 4].

2 Brief description of the data

```
1 image_files = sorted(glob.glob('training/image_2/*.png'))
2 label_files = sorted(glob.glob('training/label_2/*.txt'))
3
4 print(len(image_files), len(label_files))
5
6 image = Image.open(image_files[0])
7 print(image.mode, image.size)
```

```
7481 7481
RGB (1224, 370)
```

The dataset consists of two folders containing files. One folder contains 7,481 RGB image files measuring 1224 by 370 pixels, while the other contains 7,481 files with labels and object coordinates.

Since the labels and coordinates are stored in separate files for each image, rather than in the familiar .csv format, I'll need to create a simple function to read the file and convert it to a pandas dataframe (Listing 1). This will make it easier to conduct Exploratory Data Analysis on the objects in the images.

Listing 1. Parsing KITTI label files into a pandas dataframe.

```
1 def parse_label_file(file):
2     rows = []
3     with open(file) as f:
4         for line in f:
5             parts = line.split()
6             rows.append({
7                 'image_id': file.split('/')[-1][:4],
8                 'type': parts[0],
9                 'truncation': float(parts[1]),
10                'occlusion': int(parts[2]),
11                'alpha': float(parts[3]),
12                'xmin': float(parts[4]),
13                'ymin': float(parts[5]),
14                'xmax': float(parts[6]),
15                'ymax': float(parts[7])
16            })
17     return rows
18
19 all_rows = []
20 for f in label_files:
21     all_rows.extend(parse_label_file(f))
22 df = pd.DataFrame(all_rows)
23 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 51865 entries, 0 to 51864
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0   image_id    51865 non-null  object
1   type        51865 non-null  object
2   truncation  51865 non-null  float64
3   occlusion   51865 non-null  int64
4   alpha       51865 non-null  float64
5   xmin       51865 non-null  float64
6   ymin       51865 non-null  float64
7   xmax       51865 non-null  float64
8   ymax       51865 non-null  float64
dtypes: float64(6), int64(1), object(2)
memory usage: 3.6+ MB
```

Table 1. First rows of the parsed dataset (`df.head()`).

image_id	type	truncation	occlusion	alpha	xmin	ymin	xmax	ymax
000000	Pedestrian	0.0	0	-0.20	712.40	143.00	810.73	307.92
000001	Truck	0.0	0	-1.57	599.41	156.40	629.75	189.25
000001	Car	0.0	0	1.85	387.63	181.54	423.81	203.12
000001	Cyclist	0.0	3	-1.65	676.60	163.95	688.98	193.93
000001	DontCare	-1.0	-1	-10.00	503.89	169.71	590.61	190.13

```
1 print('types:      ', ', '.join(df['type'].unique()))
2
3 print('duplicated:', sum(df[['image_id', 'type', 'xmin', 'ymin', 'xmax', 'ymax']].duplicated()))
```

```
types:      Pedestrian, Truck, Car, Cyclist, DontCare, Misc, Van, Tram, Person_sitting
duplicated: 0
```

I ended up with a dataset of 51,865 entries (remember, there were almost ten times fewer files (and images)), each entry has 9 features (Table 1):

- `image_id` — ID of the image to which the object belongs
- `type` — object type; there are nine different types in total
- `truncation` — how much the object is cut off by the edge of the frame
- `occlusion` — how much the object is obscured by another object
- `alpha` — object direction (front relative to the camera, sideways, etc.)
- `xmin`, `ymin`, `xmax`, and `ymax` — coordinates of the object's boundaries

The dataset does not contain missing values or duplicates.

```
1 # Add some features
2
3 df['width'] = df['xmax'] - df['xmin']
4 df['height'] = df['ymax'] - df['ymin']
```

```

5 df['area'] = df['width'] * df['height']
6 df['aspect_ratio'] = df['width'] / df['height']

```

3 Exploratory Data Analysis (EDA)

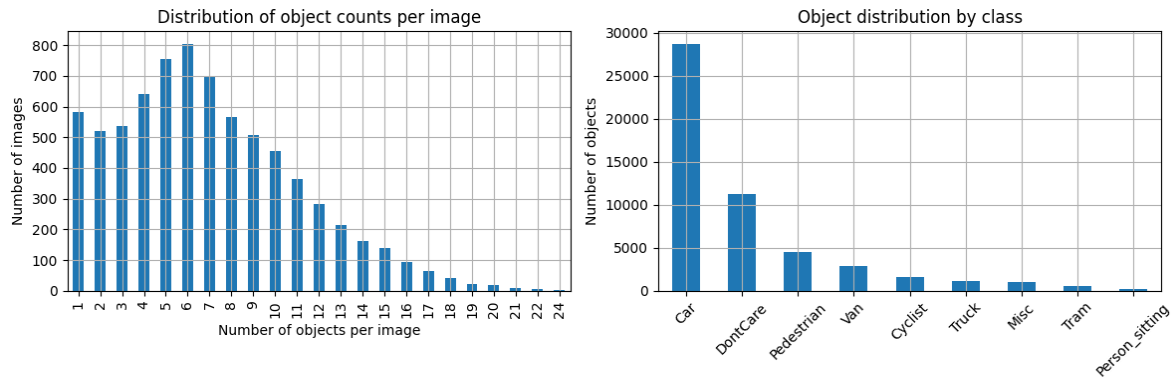


Figure 2. Distribution of object counts per image (left) and object distribution by class (right). Mean number of objects per image: 6.93.

The graphs show that each image contains at least one object, with an average of seven objects per image. The most common objects are *Car*, followed by *DontCare*, which, judging by its name, isn't the most interesting to me, while the others are significantly fewer.

Next, I'll analyze the objects' bounding boxes, starting with their sizes and areas.

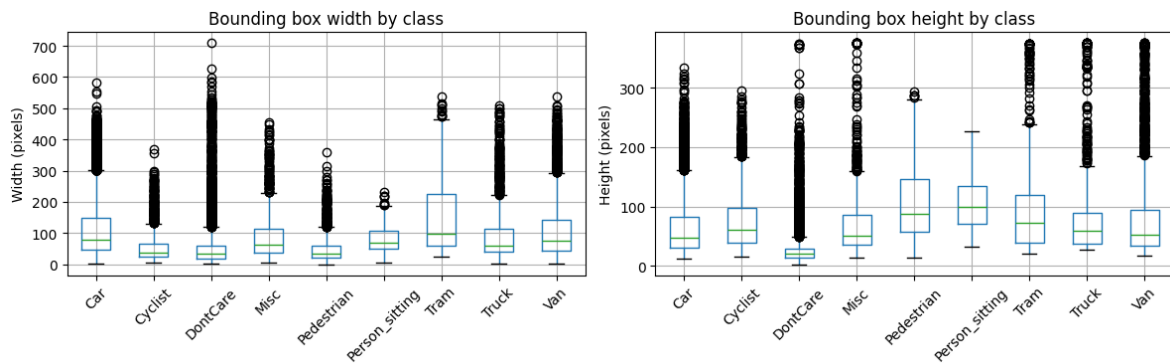


Figure 3. Bounding box width and height by class.

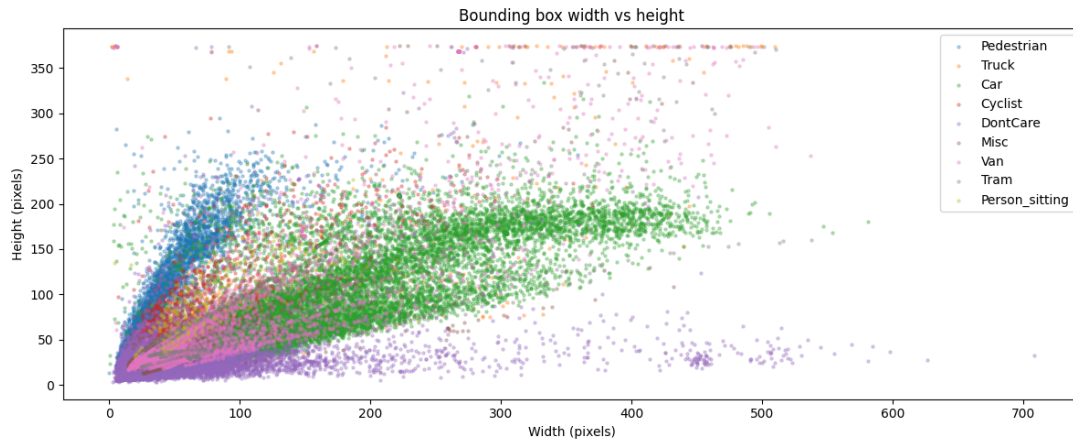


Figure 4. Bounding box width versus height for all classes.

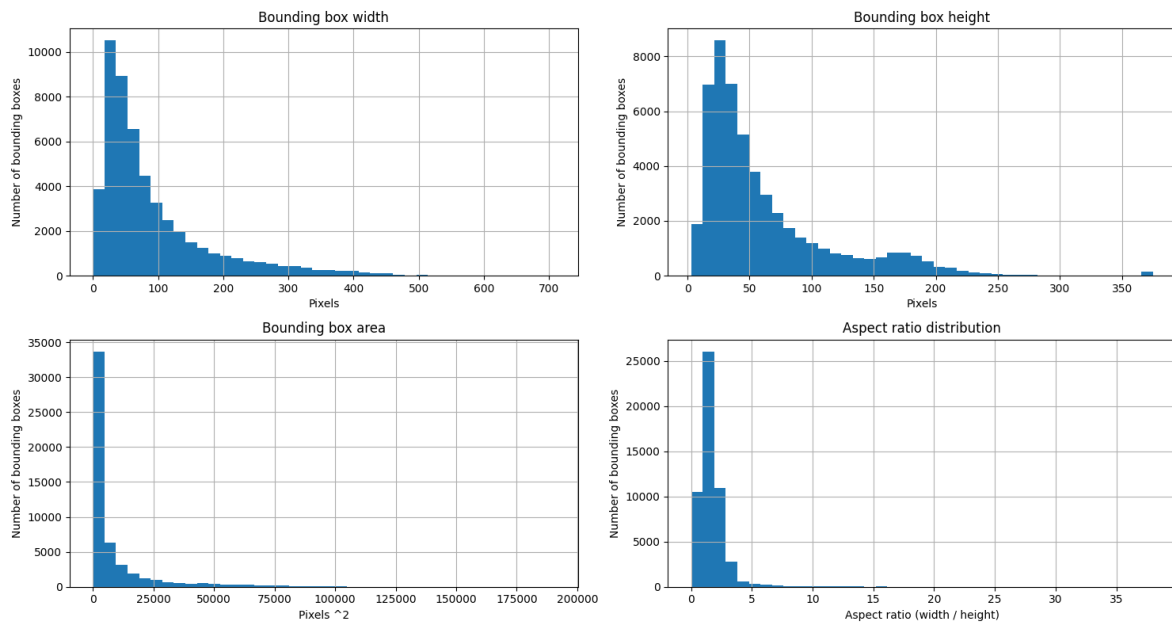


Figure 5. Distributions of bounding box width, height, area and aspect ratio.

You can see a fairly large variation in both the height/width of the bounding boxes and the aspect ratio. The size variation is clear: if an object is close, it appears larger than if it's far away. The aspect ratio variation is most likely due to the objects being too different: the tram is a horizontal, elongated rectangle from the side, while the pedestrian is vertical.

4 Focus on a single class (Car)

To simplify the model, and especially the loss function (it will have to calculate the error not only in class prediction for several objects in the image but also in the coordinates of these objects), I decided to leave only one class, **Car**. This is especially true since it accounts for the vast majority of objects in the images. In this case, the loss function will only need to calculate the binary classification error for objects and their coordinate prediction.

I'll filter out objects of the **Car** class and check for images that lack this class.

```
1 df_cars = df[df['type']=='Car'].copy()
2
3 print('Number of images without cars:', len(image_files) - len(df_cars['image_id'].unique()))
```

```
Number of images without cars: 797
```

Now there's only one class left, but 797 images don't contain a single object of that class. I could probably keep them, but the original dataset didn't include empty images that didn't contain a single object of any class. I don't want to change this condition, so I'll exclude these images from the dataset. More precisely, I'll recreate a list of image files containing only those that contain the required class.

```
1 image_files = sorted([f"training/image_2/{id}.png" for id in df_cars['image_id'].unique()])
2 label_files = sorted([f"training/label_2/{id}.txt" for id in df_cars['image_id'].unique()])
3 print(len(image_files), len(label_files))
```

```
6684 6684
```

```
1 # add feature 'distance'
2
3 q1 = df_cars['area'].quantile(0.33)
4 q2 = df_cars['area'].quantile(0.66)
5
6 def dist_group(a):
7     if a < q1: return 'far'
8     if a < q2: return 'mid'
9     return 'near'
10
11 df_cars['distance'] = df_cars['area'].apply(dist_group)
```

I added another feature, *distance*, based on the area of an object's bounding box. Since the dataset only contains *Car* objects, we can assume they are roughly comparable in size (not exactly, but this feature is only for exploratory data analysis and is not used in model training). The smaller the area of an object, the further away it is, and conversely, the larger it is, the closer it is.

5 Single-Class Exploratory Data Analysis (Car)

Now, since the dataset has changed significantly, especially in the sizes and aspect ratios of objects, I will run **Exploratory Data Analysis** again, but this time only for one class.

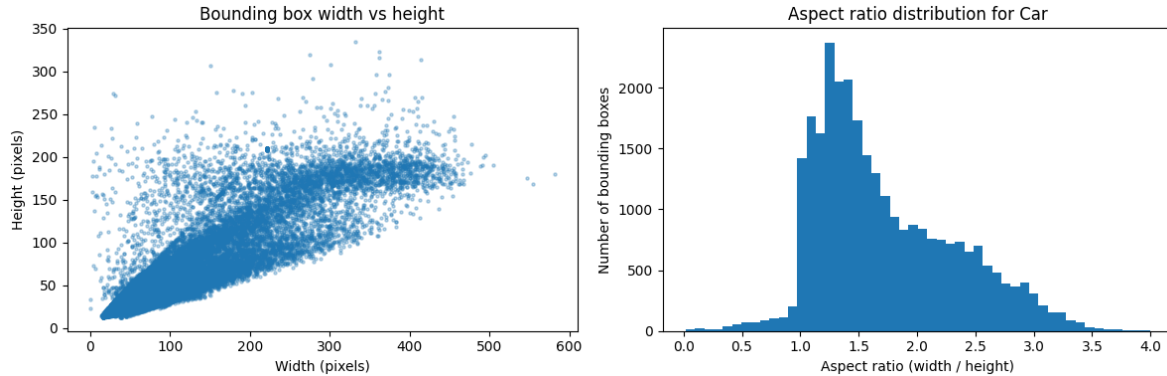


Figure 6. Bounding box width versus height and aspect ratio distribution for the *Car* class.

As you can see, the range of bounding box sizes, and especially their aspect ratio, has noticeably decreased, which is better for model training. When creating the *df_cars* dataset, I added the *distance* feature, and now you can see the distribution of objects in the image based on their distance. It's a kind of heatmap of objects in the image.

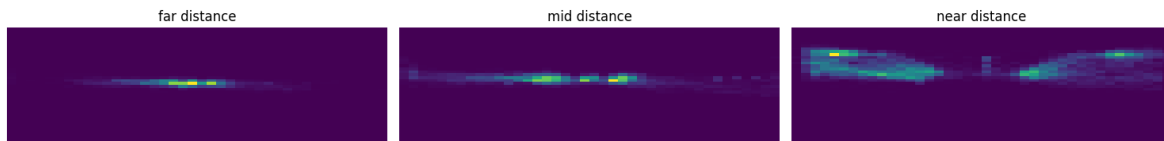


Figure 7. Heatmaps of bounding box centers for far, mid and near cars.

The heatmaps show that distant objects (small-area objects) are predominantly located in the center of the image, while nearby objects (large-area objects) are located closer to the edges. Most likely, most of the images are straight sections of road, not intersections, where nearby objects are cars in adjacent lanes, and distant objects are cars far ahead.

Interestingly, at close range, it appears as if there are more objects on the left than on the right. Let's check.

Indeed, there are more objects on the left side of the image than on the right. This is most likely due to the oncoming traffic lane, meaning even if we're driving in the far left lane, there will still be cars on the left.

The median aspect ratio of the bounding boxes doesn't change much at different distances, but their spread does.

The polar plot of the objects' orientation relative to the camera shows that we see most cars from the front or back, but not directly, but slightly to the side, as the cars are in different lanes. This further confirms that most of the images are straight sections of road without intersections.

It's probably time to look at the images themselves and their bounding boxes.

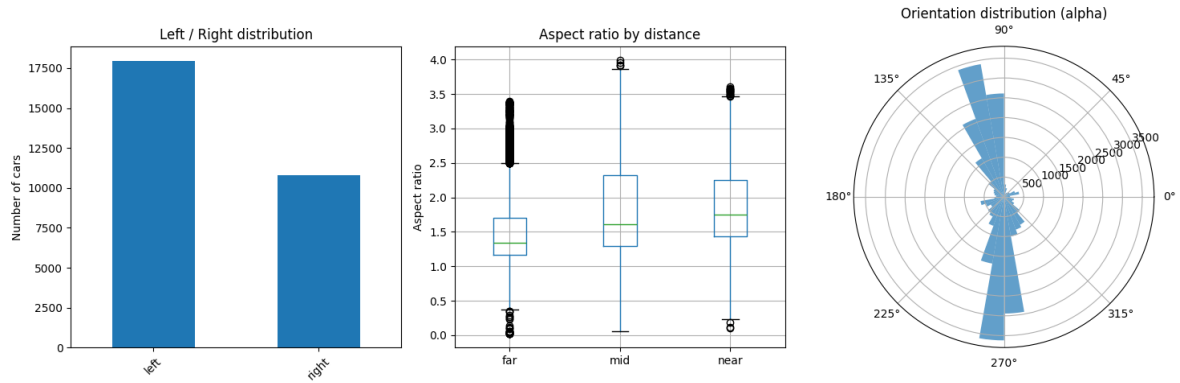


Figure 8. Left / right distribution of cars, aspect ratio by distance, and orientation (alpha) distribution.



Figure 9. Random images from the dataset with ground-truth bounding boxes of the *Car* class.

6 Plan of model architecture and learning based on EDA

After EDA, I already have a pretty good idea of what the images and objects in them are. Now the fun begins. I need to decide on the model architecture and how to prepare the training data. Speaking of data, we usually have training data (in this project, the image itself) and a target the model needs to learn to predict. But what is the target in this case?

I borrowed the idea from the popular YOLO object detection model [2]. Or rather, a greatly simplified

idea, but one that should nonetheless help solve the problem.

Concept (a highly simplified YOLO example):

- the entire image is divided into small segments (grid cells)
- a segment may contain the center of an object's bounding box (the model predicts the probability of this)
- for each object's bounding box, the model predicts the exact position of its center relative to the segment and the size of the bounding box

In other words, I divide the image into $N \times M$ small segments (a grid), and each segment contains a vector of five values: the probability of the box center, the coordinates of the box center relative to the segment, and the box size. The resulting **three-dimensional matrix** (tensor) is $N \times M \times 5$ in size, and this will be **the target the model must predict**.

The images in the dataset are 1224 x 370 pixels in size, and the most suitable cell (segment) size is **32 x 32 pixels**. Therefore, I need to resize the images to **1248 x 384 pixels** to obtain a grid of **39 x 12 cells** of 32 pixels each.

The model will consist of seven convolutional layers and five max pooling layers, which will transform a 1248 x 384 x 3 image into a 39 x 12 x 5 tensor.

I'll explain the implementation details of the loss function next to its code, as it's inconvenient to explain it so far away from the code.

7 Data preprocessing and transformation

Listing 2. Rescaling bounding boxes to the new image size and encoding them relative to the 32-pixel grid cells.

```

1 # resize the bounding box to fit the new image size
2 df_cars['rxmin'] = df_cars['xmin'] * 1248 / 1224
3 df_cars['rxmax'] = df_cars['xmax'] * 1248 / 1224
4 df_cars['rymin'] = df_cars['ymin'] * 384 / 370
5 df_cars['rymax'] = df_cars['ymax'] * 384 / 370
6
7 # size and center of the bounding box
8 resized_width = df_cars['rxmax'] - df_cars['rxmin']
9 resized_height = df_cars['rymax'] - df_cars['rymin']
10 resized_cx = (df_cars['rxmin'] + df_cars['rxmax']) / 2.0
11 resized_cy = (df_cars['rymin'] + df_cars['rymax']) / 2.0
12
13 # cell indices
14 df_cars['cell_j'] = (resized_cx / 32).astype(int).clip(0, 38) # column
15 df_cars['cell_i'] = (resized_cy / 32).astype(int).clip(0, 11) # row
16
17 # x, y of center of bounding box inside cell
18 df_cars['bx'] = resized_cx / 32 - df_cars['cell_j']
19 df_cars['by'] = resized_cy / 32 - df_cars['cell_i']
20
21 # bounding box width and height as log scale

```

```

22 df_cars['bw'] = np.log(resized_width / 32 + 1e-9)
23 df_cars['bh'] = np.log(resized_height / 32 + 1e-9)
24
25 df_cars.head()

```

Table 2. Encoded bounding boxes (selected columns of `df_cars.head()`).

image_id	rxmin	rxmax	rymin	rymax	cell_j	cell_i	bx	by	bw	bh
000001	395.23	432.12	188.41	210.81	12	6	0.9274	0.2377	0.1422	-0.3568
000002	670.28	713.80	197.32	231.84	21	6	0.6262	0.7057	0.3074	0.0758
000003	626.28	741.57	188.66	295.55	21	7	0.3727	0.5657	1.2817	1.2060
000004	285.88	351.66	192.10	223.75	9	6	0.9616	0.4977	0.7207	-0.0112
000004	372.30	414.07	191.52	212.96	12	6	0.2871	0.3201	0.2665	-0.4004

I'll try rendering a scaled image with a grid and bounding box for one of the cars to ensure the data was transformed successfully. I'll separate image loading and grid rendering into separate functions, as I'll need them frequently (Listing 23 in Appendix A).

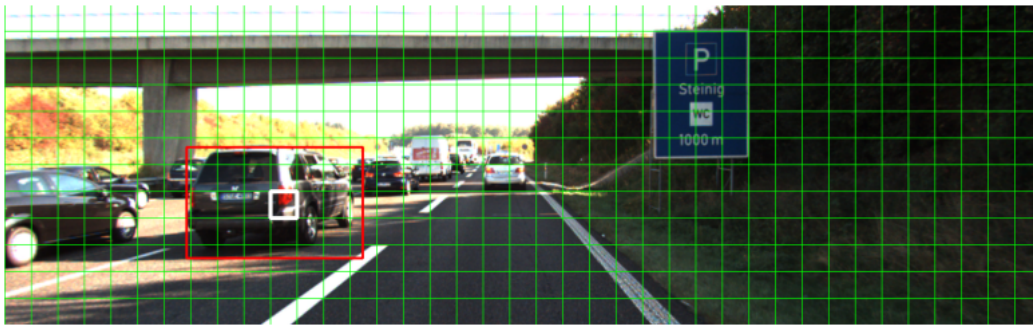


Figure 10. Resized image 000047 with the 39×12 grid. The white square marks the cell containing the center of the bounding box; the red frame is the bounding box decoded back from the target values.

The transformed data was successfully decoded. We can see the image divided into 468 cells (39×12) each measuring 32 pixels. For this example, I've selected a single object; the white square marks the cell containing the center of the object's (car's) bounding box, and the red frame marks the bounding box itself.

Now we need to create a **data generator function** (images and targets) for training and validating the model. The generator contains an infinite loop in which it creates batches of images and targets for training the model. Each batch contains several resized and normalized (0..1) images and a three-dimensional tensor of size $12 \times 39 \times 5$ (unlike images, where width comes first, then height, tensors usually have rows first, then columns) for each image. This is the same grid whose cells contain the bounding box information we discussed above (Listing 3).

Listing 3. Target encoding and the data generator.

```

1 # I moved it into a separate function, since I will use it again later
2 def load_image_resize_norm(id):
3     return load_image_resize(id).astype('float32') / 255.0
4

```

```

5 # I moved it into a separate function, since I will use it again later
6 def encode_target(id):
7     target = np.zeros((12, 39, 5), dtype=np.float32)
8
9     for row in df_cars[df_cars['image_id']==id].itertuples():
10         i, j = row.cell_i, row.cell_j
11         if target[i, j, 4] == 1.0: continue # there is already an object in this
            cell - ignore
12         target[i, j] = [row.bx, row.by, row.bw, row.bh, 1.0] # 5 parameters in each cell
13     return target
14
15
16 def data_generator(ids, batch_size, shuffle=True):
17
18     n = len(ids)
19     while True:
20         if shuffle:
21             np.random.shuffle(ids)
22
23         for start in range(0, n, batch_size):
24             batch_ids = ids[start:min(start + batch_size, n)]
25
26             batch_imgs = []
27             batch_targets = []
28
29             if len(batch_ids) == 0:
30                 continue
31
32             for id in batch_ids:
33                 batch_imgs.append(load_image_resize_norm(id))
34                 batch_targets.append(encode_target(id))
35
36             batch_imgs = np.stack(batch_imgs, axis=0).astype(np.float32)
37             batch_targets = np.stack(batch_targets, axis=0).astype(np.float32)
38
39             yield batch_imgs, batch_targets

```

```

1 train, val = train_test_split(df_cars['image_id'].unique(), test_size=0.1, random_state=42)
2 print('train:', len(train), 'val:', len(val))

```

```
train: 6015 val: 669
```

I've split all the images (or rather, image identifiers) into training and validation sets. Now on to the model itself.

8 Model implementation

I also borrowed the model's architecture from early versions of YOLO [2, 3] and simplified it for my purposes. It includes **six convolutional layers**, each with **batch normalization** [8], **ReLU activation**, and **pooling**. With each block, the **image size is reduced by a factor of four** (halving the width and height), and the number of filters doubles. Ultimately, the image is convolved to **12 x 39**, which is the number of grid cells.

Finally, there's another convolutional layer with five filters, one for each output (object presence + bounding box center coordinates in the cell + bounding box height and width). The model's output layer has no activation function, meaning it returns **raw logits**, since different outputs will be activated differently later. Ultimately, the **model will return 12 x 39 x 5**, or **2,340 outputs** (Listing 4).

Listing 4. Model architecture: six convolutional blocks and a 1×1 output convolution with five filters per grid cell.

```

1  def build_model():
2      return tf.keras.Sequential([
3          layers.Input(shape=(384, 1248, 3)),
4
5          layers.Conv2D(16, 3, 1, 'same', use_bias=False, kernel_initializer='he_normal'),
6          layers.BatchNormalization(),
7          layers.LeakyReLU(alpha=0.1),
8          layers.MaxPooling2D(2, 2), # 192x624
9
10         layers.Conv2D(32, 3, 1, 'same', use_bias=False, kernel_initializer='he_normal'),
11         layers.BatchNormalization(),
12         layers.LeakyReLU(alpha=0.1),
13         layers.MaxPooling2D(2, 2), # 96x312
14
15         layers.Conv2D(64, 3, 1, 'same', use_bias=False, kernel_initializer='he_normal'),
16         layers.BatchNormalization(),
17         layers.LeakyReLU(alpha=0.1),
18         layers.MaxPooling2D(2, 2), # 48x156
19
20         layers.Conv2D(128, 3, 1, 'same', use_bias=False, kernel_initializer='he_normal'),
21         layers.BatchNormalization(),
22         layers.LeakyReLU(alpha=0.1),
23         layers.MaxPooling2D(2, 2), # 24x78
24
25         layers.Conv2D(256, 3, 1, 'same', use_bias=False, kernel_initializer='he_normal'),
26         layers.BatchNormalization(),
27         layers.LeakyReLU(alpha=0.1),
28         layers.MaxPooling2D(2, 2), # 12x39
29
30         layers.Conv2D(512, 3, 1, 'same', use_bias=False, kernel_initializer='he_normal'),
31         layers.BatchNormalization(),
32         layers.LeakyReLU(alpha=0.1),
33
34         # Output 12 x 39 x [bx, by, bw, bh, obj]
35         layers.Conv2D(5, 1, 1, 'same', kernel_initializer='he_normal'),
36     ])

```

8.1 Loss function

I had to write my own **loss function** because the model outputs five parameters for each cell. They are activated differently, and the error is calculated differently for each. Let's go over them:

- The first two outputs are the relative coordinates of the center of the bounding box in the cell. Sigmoid activation is used, as it is most suitable for values from 0 to 1 (relative coordinates).
- The next two outputs are the width and height of the bounding box, as the logarithm of the scale; they are simply clipped from -4 to 4 .
- The last output is the presence or absence of the center of the object in the cell, objectness, essentially a binary classification, using sigmoid activation.

For coordinates, the loss is calculated as MSE, for objectness as Cross-entropy, then everything is summed up and returned as the total loss (Listing 5).

Listing 5. Custom loss function combining the coordinate (MSE) and objectness (cross-entropy) terms, weighted by `lambda_coord` and `lambda_noobj`.

```

1  def make_loss(lambda_coord=5.0, lambda_noobj=2.0):
2      def loss_fn(y_true, y_pred):
3
4          # bx, by via sigmoid; bw, bh just clip
5          pred_xywh = tf.concat([
6              tf.sigmoid(y_pred[..., 0:2]),
7              tf.clip_by_value(y_pred[..., 2:4], -4.0, 4.0),
8          ], axis=-1)
9
10         obj_mask = y_true[..., 4]
11         noobj_mask = 1.0 - obj_mask
12
13         # coordinate loss
14         coord_sq = tf.reduce_sum(tf.square(pred_xywh - y_true[..., 0:4]), axis=-1) # (B, 12, 39)
15         coord_loss = tf.reduce_sum(obj_mask * coord_sq) / (tf.reduce_sum(obj_mask) + 1e-6)
16
17         # objectness loss
18         obj_loss = tf.nn.sigmoid_cross_entropy_with_logits(
19             labels=y_true[..., 4],
20             logits=y_pred[..., 4]
21         )
22
23         obj_loss_obj = tf.reduce_sum(obj_mask * obj_loss) / (tf.reduce_sum(obj_mask) + 1e-6)
24         obj_loss_noobj = tf.reduce_sum(noobj_mask * obj_loss) / (tf.reduce_sum(noobj_mask) + 1e-6)
25
26         total_loss = lambda_coord * coord_loss + obj_loss_obj + lambda_noobj * obj_loss_noobj
27         return total_loss
28     return loss_fn

```

9 Hyperparameters tuning

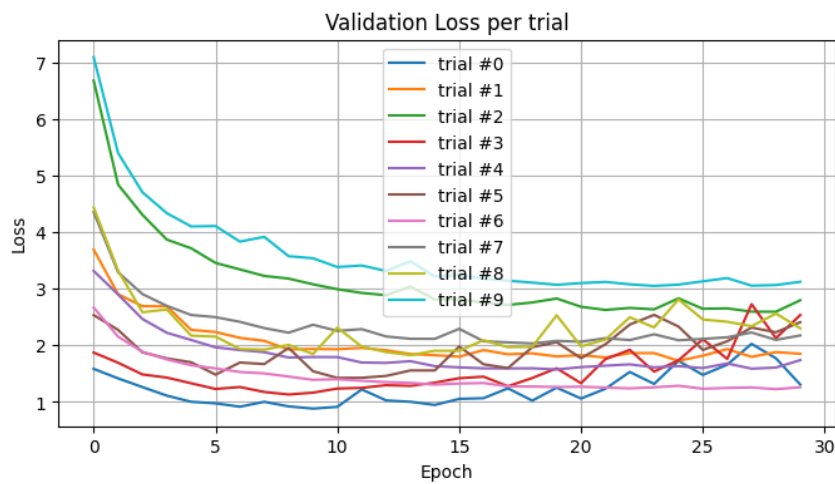
First, I will select the optimal parameters for the model using Random Search [9] (Listing 6).

Listing 6. Random Search over the learning rate and the loss weights, 10 trials of 30 epochs each (about 37 minutes per trial).

```
1  BATCH_SIZE = 4
2
3  random.seed(42)
4  np.random.seed(42)
5  tf.random.set_seed(42)
6
7  results = []
8
9  for trial in range(10):
10     start = time.time()
11
12     # random hyperparameters for model and loss function
13     lr = 10 ** random.uniform(math.log10(1e-5), math.log10(3e-4))
14
15     lambda_coord = random.uniform(2.0, 10.0)
16     lambda_noobj = random.uniform(0.5, 4.0)
17
18     # a new model for every trial
19     model = build_model()
20
21     model.compile(optimizer=tf.keras.optimizers.Adam(lr),
22                  loss=make_loss(lambda_coord, lambda_noobj)) # loss with hyperparameters
23
24
25     history = model.fit(
26         data_generator(train, BATCH_SIZE),
27         steps_per_epoch=len(train) // BATCH_SIZE,
28         epochs=30,
29         validation_data=data_generator(val, BATCH_SIZE, shuffle=False),
30         validation_steps=max(1, len(val) // BATCH_SIZE),
31         verbose=0
32     )
33
34     # save configuration, history, best epoch, and val_loss
35     results.append({
36         'config': {'lr': lr, 'lambda_coord': lambda_coord, 'lambda_noobj': lambda_noobj},
37         'history': history.history,
38         'best_epoch': np.argmin(history.history['val_loss']),
39         'best_val_loss': min(history.history['val_loss']),
40     })
41
42     print(f'Trial #{trial} done, elapsed time {(time.time() - start) / 60:.2f} minutes')
```

Table 3. Random Search results: hyperparameters, best epoch and best validation loss of each trial.

trial	lr	lambda_coord	lambda_noobj	best_epoch	best_val_loss
0	0.000088	2.200086	1.462603	9	0.883622
1	0.000064	6.723940	0.611239	24	1.727083
2	0.000020	7.199076	2.407295	28	2.594124
3	0.000132	3.277275	1.979150	8	1.133028
4	0.000036	4.871835	1.703845	19	1.575053
5	0.000274	5.028275	2.432142	10	1.425972
6	0.000019	2.556441	2.814422	28	1.226898
7	0.000036	5.627282	3.419386	18	2.034484
8	0.000242	7.184283	2.631959	13	1.827309
9	0.000025	9.403129	2.908567	23	3.049865

**Figure 11.** Validation loss per trial.

As you can see from the table and graphs, the smallest loss was for **trial #0** at epoch 9 with parameters `lr=0.000088`, `lambda_coord=2.200086`, and `lambda_noobj=1.462603`. Then the model begins to overfit, and the validation loss begins to increase.

10 Training and evaluating

I'll make a separate graph of this trial, and also recreate the model with optimal parameters, train it for 9 epochs, and perform inference for the validation sample (Listing 7).

Listing 7. Training the final model with the best hyperparameters and running inference on the validation set.

```

1 random.seed(42)
2 np.random.seed(42)
3 tf.random.set_seed(42)
4
5 model = build_model()
6 model.compile(optimizer=tf.keras.optimizers.Adam(best_trial['config']['lr']),
7               loss=make_loss(best_trial['config']['lambda_coord'],
8                             best_trial['config']['lambda_noobj']))

```

```

8
9 history = model.fit(
10     data_generator(train, BATCH_SIZE),
11     epochs=best_trial['best_epoch'],
12     steps_per_epoch=len(train) // BATCH_SIZE,
13     validation_data=data_generator(val, BATCH_SIZE, shuffle=False),
14     validation_steps=max(1, len(val) // BATCH_SIZE),
15     verbose=0
16 )
17
18 pred, target = [], []
19 for id in val:
20     pred.append(model.predict(load_image_resize_norm(id)[None, ...], verbose=0)[0])
21     target.append(encode_target(id))
22
23 # convert lists into arrays
24 val_pred = np.stack(pred, axis=0)
25 val_target = np.stack(target, axis=0)
26
27 def sigmoid(z):
28     return 1.0/(1.0+np.exp(-z))
29
30 true_objects = val_target[..., 4].reshape(-1) # N * 12 * 39
31 pred_objects = sigmoid(val_pred[..., 4]).reshape(-1) # N * 12 * 39

```

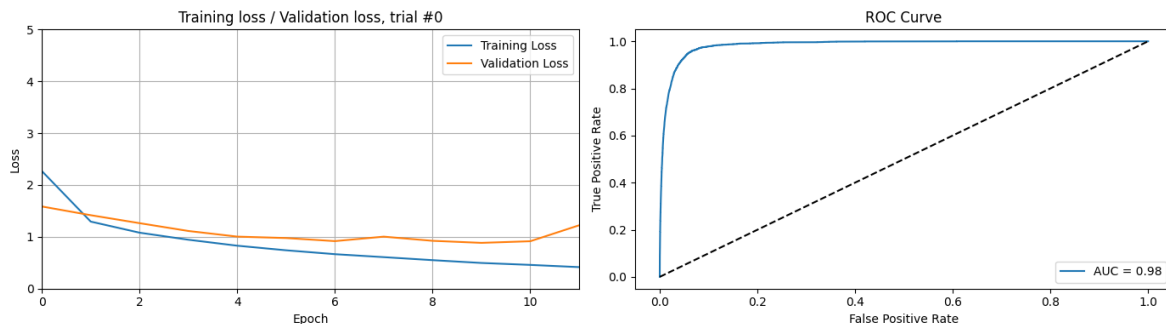


Figure 12. Training and validation loss of trial #0 (left) and the ROC curve of the objectness prediction on the validation set (right).

Well, both graphs look quite good, but I suspect this is due to a severe imbalance, as the grid cells without cars in the image are incomparably larger than the cells with cars. That is, even if the model hadn't predicted a single car, the True Negative would still be so high that it would dominate any errors. So, graphs aside, let's look at what we get in the images.

10.1 What the model predicts

But first, let's figure out what the model actually predicted.

```
1 print(val_pred.shape)
```

```
(669, 12, 39, 5)
```

A four-dimensional tensor, where 669 is the number of images in the validation set, 12 x 39 is the number of cells in the image grid, and 5 is the prediction vector for a single cell in this grid. The first four elements of this vector correspond to the bounding box, and the last one corresponds to the presence or absence of the object's (car's) center in the cell.

I'll take one image (the same one we looked at before training the model) and explain it using it.

```
1 pred_single_image = model.predict(load_image_resize_norm('000047')[None, ...], verbose=0)[0]
2 pred_single_image.shape
```

```
(12, 39, 5)
```

As I said, the last element in the prediction vector is the prediction of the presence or absence of the object's center in the cell, but this is a **logit**, and to get the probability you need to apply the **sigmoid** function to it.

```
1 objects = sigmoid(pred_single_image[..., 4])
2 objects.shape
```

```
(12, 39)
```

Now all that remains is the predictions for each grid cell of the image, and they can be output as a heatmap (Figure 13a).

Typically, a threshold of 0.5 is used for sigmoid results. I will set all values below this threshold to zero, and leave everything above as probabilities, so that it reflects how the model estimates the probability of the object's center in each cell (Figure 13b).

```
1 objects[objects < 0.5] = 0
```



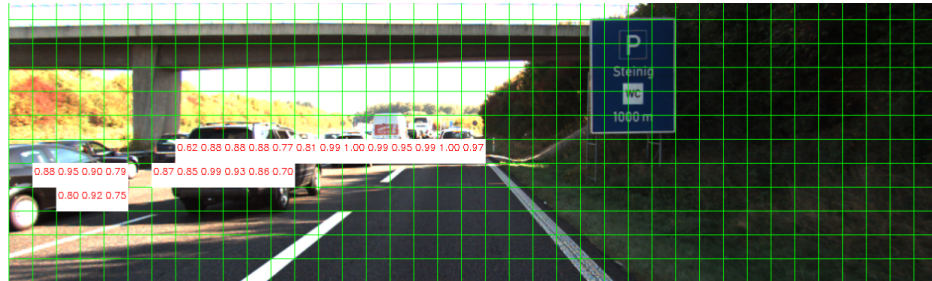
(a) Raw sigmoid outputs



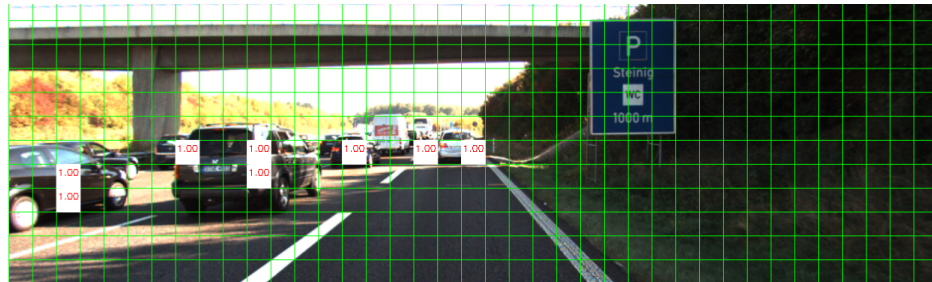
(b) Values below 0.5 set to zero

Figure 13. Predicted objectness for each of the 12×39 grid cells of image 000047.

It's cleaner now, and you can overlay the relevant probabilities directly on top of the grid image to make it clearer what exactly is in the image in those cells where the model predicted objects.



(a) Predicted objects (threshold 0.5)



(b) True objects

Figure 14. Grid cells in which the model predicted the center of a car's bounding box, compared with the ground truth.

10.2 Choosing the threshold

The top image shows the model's predicted object centers, while the bottom image shows the true objects. It's clear that the model is working, with some objects being accurately predicted, but overall, the model is significantly **overestimating**, finding too many objects, and the default threshold value of 0.5 doesn't seem appropriate. It's worth increasing the threshold value, but not randomly, but by determining which threshold value yields the best **F1 Score** and **Matthews Correlation Coefficient (MCC)** [10] metrics.

To do this, I will make a graph of the F1 and MCC metrics values depending on the threshold value.

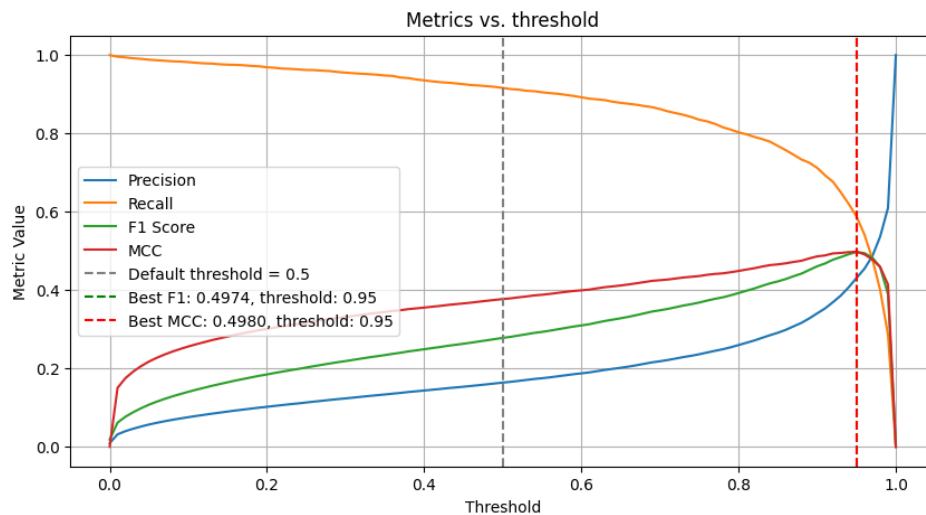


Figure 15. Precision, Recall, F1 Score and MCC of the objectness prediction depending on the threshold.

Both the F1 score and MCC metrics insist that the **best threshold** value is **0.95**. This initially seems alarming, but given the strong imbalance in favor of True Negative, it seems appropriate. Both metrics seem fairly robust to imbalanced classes, but as always, the threshold is a **tradeoff** between **Type I** and **Type II** errors. However, in my opinion, such a threshold value would make the evaluation overly conservative, worsening vehicle detection in favor of reducing false positives.

I'll show this in a confusion matrix for threshold values of 0.8, 0.85, and 0.95.

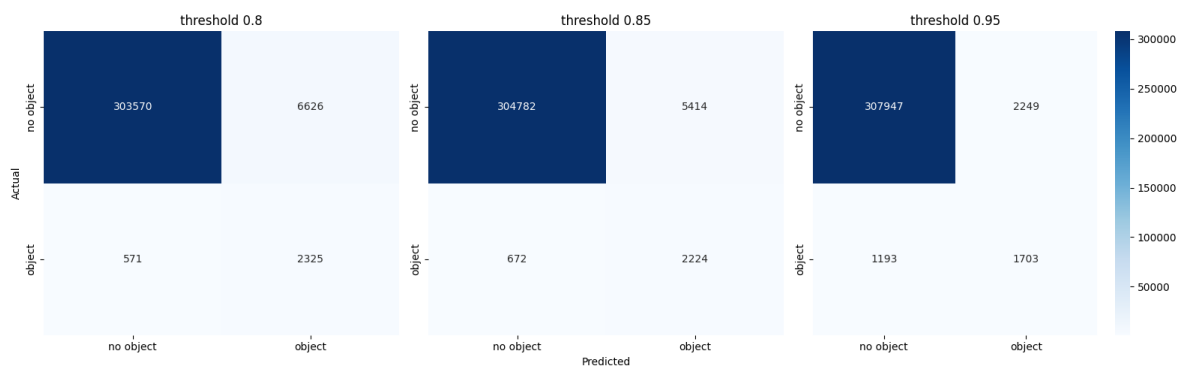


Figure 16. Confusion matrices of the objectness prediction for thresholds 0.8, 0.85 and 0.95.

You can see that increasing the threshold significantly reduces the Type I Error (False Positive), as

does the number of recognized objects. In my opinion, a threshold value of around 0.85 seems more balanced, even though it performs slightly worse in the F1 and MCC metrics (in reality, the MCC is almost identical). So, I'll use a threshold value of 0.85.

Let's look at an image with this threshold.



Figure 17. Predicted objects with the threshold raised to 0.85.

10.3 Bounding boxes and Non-Maximum Suppression

The number of objects has noticeably decreased, and I have an idea for how to reduce them even further. But first, I need to decode the remaining vector elements for each cell so that the boundary boxes can be drawn.

Since I'm decoding everything returned by the model this time, I'll decode the objectness again using sigmoid activation. I'll apply the same activation to the relative coordinates of the boundary box center inside the cell, and clip the height and width to values between -4 and 4 . Basically, it's the same as I did in the loss function (Listing 8).

Listing 8. Decoding the model output into bounding boxes in pixels.

```
1 def decode_prediction(pred, threshold=0.5):
2
3     # for each grid cell activate values
4     obj = sigmoid(pred[..., 4])      # objectness
5     bx = sigmoid(pred[..., 0])      # bounding box relative center x
6     by = sigmoid(pred[..., 1])      # bounding box relative center y
7     bw = np.clip(pred[..., 2], -4, 4) # bounding box width
8     bh = np.clip(pred[..., 3], -4, 4) # bounding box height
9
10    # active cells mask
11    mask = obj > threshold
12
13    if not np.any(mask):
14        return (np.zeros((0, 4), dtype=np.float32),
15                np.zeros((0,), dtype=np.float32),
16                np.zeros((0,), dtype=np.int32),
17                np.zeros((0,), dtype=np.int32))
18
19    # cell indices where obj > threshold
20    i, j = np.where(mask)
21
22    # restore absolute center values
```

```

23     # and bounding box dimensions in pixels
24     cx = (j + bx[mask]) * 32
25     cy = (i + by[mask]) * 32
26
27     bw = np.exp(bw[mask]) * 32
28     bh = np.exp(bh[mask]) * 32
29
30     # absolute bounding box values in pixels
31     xmin = cx - bw / 2.0
32     ymin = cy - bh / 2.0
33     xmax = cx + bw / 2.0
34     ymax = cy + bh / 2.0
35
36     boxes = np.stack([xmin, ymin, xmax, ymax], axis=-1).astype(np.float32)
37     scores = obj[mask].astype(np.float32)
38
39     return boxes, scores, i, j

```

As expected, if we had too many detected objects in the image, there would also be a lot of bounding boxes (Figure 18a). It's clear that some boxes belong to the same cars and largely overlap each other.

To address this, YOLO uses an algorithm called **Non-Maximum Suppression (NMS)**, which finds overlapping boxes, keeps the box with the highest score, and removes the rest. This algorithm, in turn, uses the **Intersection over Union (IoU)** metric, which measures the extent to which two boxes intersect.

I'll implement a slightly simplified version of this algorithm (Listing 9) and apply it to prediction. We'll see how well it removes the extra bounding boxes in the image above.

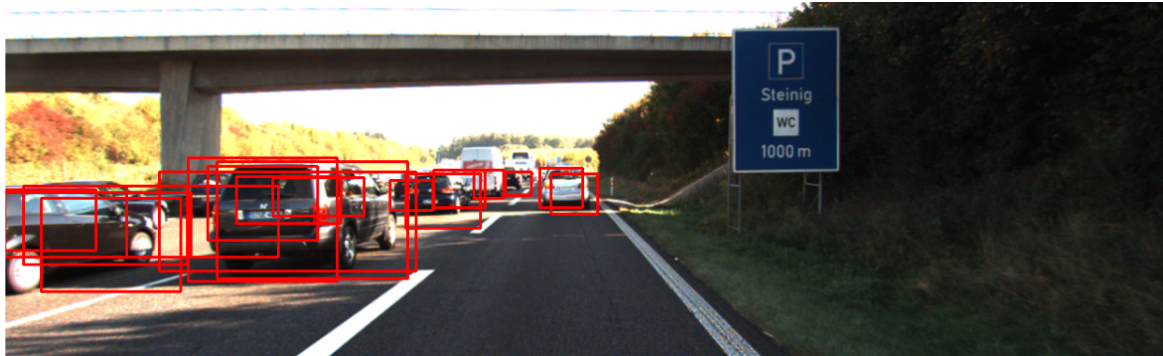
Listing 9. Intersection over Union and a simplified Non-Maximum Suppression.

```

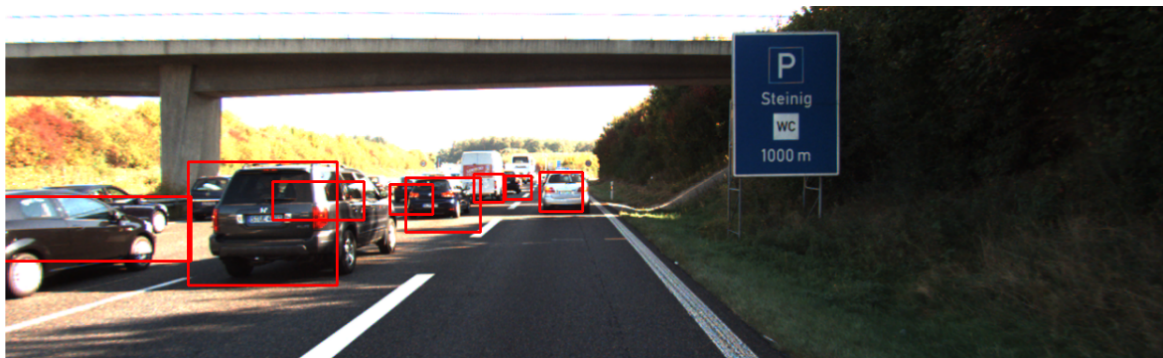
1  def iou(box1, box2):
2      x1_min, y1_min, x1_max, y1_max = box1
3      x2_min, y2_min, x2_max, y2_max = box2
4
5      inter_w = max(min(x1_max, x2_max) - max(x1_min, x2_min), 0.0)
6      inter_h = max(min(y1_max, y2_max) - max(y1_min, y2_min), 0.0)
7      inter_area = inter_w * inter_h # the interior intersection of two boxes
8
9      area1 = max(x1_max - x1_min, 0.0) * max(y1_max - y1_min, 0.0)
10     area2 = max(x2_max - x2_min, 0.0) * max(y2_max - y2_min, 0.0)
11
12     union = area1 + area2 - inter_area
13     return inter_area / (union + 1e-9)
14
15 def nms(boxes, scores, iou_thr=0.5):
16     if len(boxes) == 0:
17         return np.array([], dtype=np.int32)
18     idxs = np.argsort(-scores)
19     keep = []
20     while len(idxs) > 0:
21         i = idxs[0]
22         keep.append(i)
23         if len(idxs) == 1:
24             break

```

```
25     rest = idxs[1:]
26     ious = np.array([iou(boxes[i], boxes[j]) for j in rest])
27     idxs = rest[ious < iou_thr]
28     return np.array(keep, dtype=np.int32)
29
30 keep = nms(boxes, scores, iou_thr=0.2)
```



(a) Decoded bounding boxes, threshold 0.85



(b) After Non-Maximum Suppression, IoU threshold 0.2

Figure 18. Effect of Non-Maximum Suppression on the predicted bounding boxes.

The bounding boxes have become smaller, and the detection is much cleaner. It looks like NMS has done the job, although the overlap threshold had to be lowered to 0.2. But anyway, it now looks more like the detected objects, and the bounding boxes generally more or less match the vehicles.

Let's look at the prediction and NMS results in a few more images (Figure 19).



Figure 19. Predictions of my model after NMS on five random images from the validation set.

11 Alternative model: pre-trained YOLO

As an alternative, I want to try a real YOLO v8 object detection model [4, 5]. I'll use a pre-trained version on the Common Objects in Context (COCO) dataset [6], a large-scale object detection, segmentation, and captioning dataset.

I'll further train it on my training set and compare the results. I expect it to outperform my simplified

model.

But first, I'll prepare a dataset in a format compatible with this model (Listing 10).

Listing 10. Dataset in the KerasCV bounding box format.

```

1  def make_yolo_dataset(df, batch_size=4, shuffle=True):
2      groups = df.groupby('image_id') # there can be many objects in one image
3
4      image_paths = []
5      boxes_list = []
6
7      for img_path, g in groups:
8          image_paths.append(img_path)
9          boxes = g[['rxmin', 'rymin', 'rxmax', 'rymax']].values.astype('float32')
10         boxes_list.append(boxes)
11
12     image_paths = np.array(image_paths)
13
14     def gen():
15         for path, boxes in zip(image_paths, boxes_list):
16             yield {
17                 'images': load_image_resize_norm(path),
18                 'bounding_boxes': {
19                     'boxes': boxes,
20                     'classes': np.zeros((boxes.shape[0],), dtype='float32'), # one class: 0 for
21                     all boxes
22                 },
23             }
24
25     output_signature = {
26         'images': tf.TensorSpec(shape=(384, 1248, 3), dtype=tf.float32),
27         'bounding_boxes': {
28             'boxes': tf.TensorSpec(shape=(None, 4), dtype=tf.float32),
29             'classes': tf.TensorSpec(shape=(None,), dtype=tf.float32),
30         },
31     }
32
33     ds = tf.data.Dataset.from_generator(gen, output_signature=output_signature)
34
35     if shuffle:
36         ds = ds.shuffle(buffer_size=len(image_paths))
37
38     # Ragged batch, because the number of boxes in the photo is different
39     ds = ds.ragged_batch(batch_size, drop_remainder=True)
40     ds = ds.prefetch(tf.data.AUTOTUNE)
41
42     return ds

```

Listing 11. YOLOv8 detector with a COCO pre-trained backbone and CIoU box loss [7].

```

1  # backbone with COCO pre-trained
2  backbone = keras_cv.models.YOLOV8Backbone.from_preset(
3      'yolo_v8_s_backbone_coco', # small model because I have weak 3060 card
4      load_weights=True,
5  )

```

```

6
7 yolo = keras_cv.models.YOLOV8Detector(
8     num_classes=1,          # one class: car
9     bounding_box_format='xyxy', # as in df
10    backbone=backbone,
11    fpn_depth=3,
12 )
13
14 optimizer = keras.optimizers.Adam(learning_rate=1e-5)
15
16 yolo.compile(
17     optimizer=optimizer,
18     classification_loss='binary_crossentropy',
19     box_loss='ciou', # ready CIoU-loss
20 )

```

Listing 12. Fine-tuning YOLOv8 on the training set.

```

1 EPOCHS = 30
2
3 random.seed(42)
4 np.random.seed(42)
5 tf.random.set_seed(42)
6
7 yolo_history = yolo.fit(
8     make_yolo_dataset(df_cars[df_cars['image_id'].isin(train)].copy(), batch_size=4, shuffle=True),
9     validation_data=make_yolo_dataset(df_cars[df_cars['image_id'].isin(val)].copy(),
10    batch_size=4, shuffle=False),
11    epochs=EPOCHS,
12    verbose=0
13 )

```

Listing 13. Mapping YOLO predictions onto the same 12×39 grid so that both models can be evaluated in the same way.

```

1 def yolo_decode(pred):
2     result = []
3     for n, confidence in enumerate(pred['confidence']):
4
5         cells = np.zeros((12, 39), dtype=np.float32)
6
7         for m, conf in enumerate(confidence):
8             if conf < 0: continue
9
10            box = pred['boxes'][n][m]
11            j = ((box[0] + box[2]) / 2 / 32).astype(int).clip(0, 38)
12            i = ((box[1] + box[3]) / 2 / 32).astype(int).clip(0, 11)
13            if cells[i, j] > conf: continue
14            cells[i, j] = conf
15
16        result.append({
17            'boxes': pred['boxes'][n],
18            'cells': cells,
19            'confidence': pred['confidence'][n]
20        })

```

```

21     return result
22
23 yolo_pred = []
24 for id in val:
25     yolo_pred.extend(yolo_decode(yolo.predict(load_image_resize_norm(id)[None, ...], verbose=0)))
26
27
28 yolo_objects = np.stack([pred['cells'] for pred in yolo_pred], axis=0).reshape(-1)

```

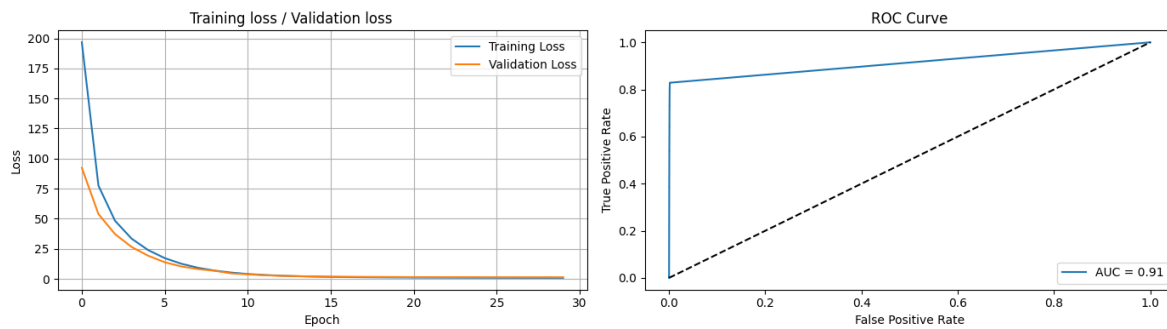


Figure 20. YOLOv8 training and validation loss (left) and the ROC curve on the validation set (right).

The loss graph looks completely different from my model's, but they **can't be directly compared**; these models have completely different loss functions.

The ROC Curve graph looks unusual, but the YOLO model includes a Non-Maximum Suppression (NMS) algorithm, similar to the one I used for post-processing on my model. And the confidence values the model produced are not quite the typical sigmoid activation results.

Therefore, before displaying anything in the images, I'll create a **F1 Score** and **MCC** graph to find the best threshold value, assuming it will be significantly different from the standard 0.5.

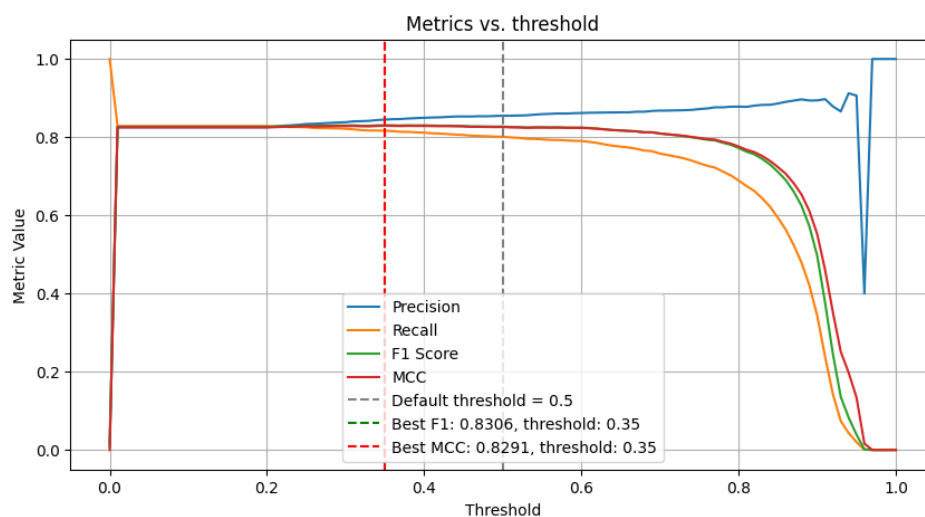
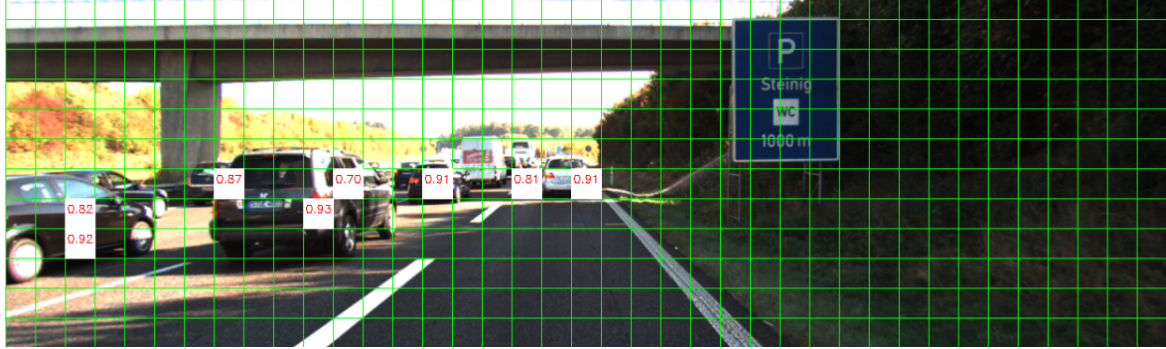


Figure 21. Precision, Recall, F1 Score and MCC of YOLOv8 depending on the confidence threshold.

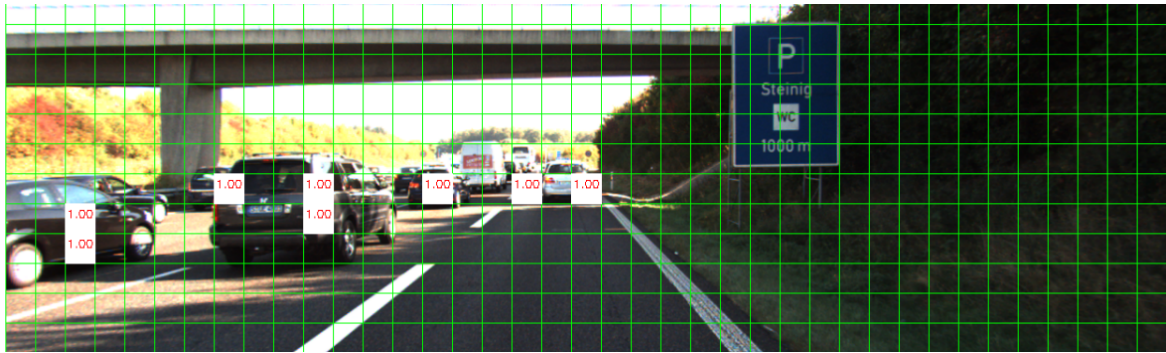
As I expected, the metric graphs look very unusual. In fact, any value other than 0 is already a potentially detected object. So, zero here is something close to the standard value of 0.5 in a regular sigmoid activation. But a regular sigmoid activation doesn't always yield the best tradeoff between

detected objects and false positives. Judging by the graph, the best ratio, and therefore the best F1 Score and MCC metrics, are achieved with a **threshold value of 0.35**.

Using this value, I'll plot the detected bounding box centers on the image and compare them with the true values (Figure 22). Then, I'll plot the resulting bounding boxes on the image (Figure 23).



(a) YOLO objects (threshold 0.35)



(b) True objects

Figure 22. Grid cells containing the centers of the bounding boxes predicted by YOLOv8, compared with the ground truth.

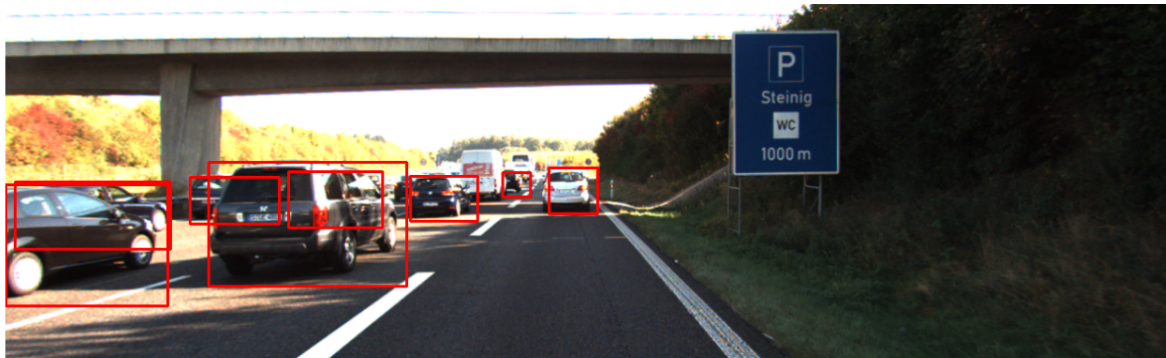


Figure 23. Bounding boxes predicted by YOLOv8, confidence threshold 0.35.

12 Model comparison

Although it is obvious from the image that YOLO, as expected, performed better than my simplified model, I need graphs and numbers for comparison.

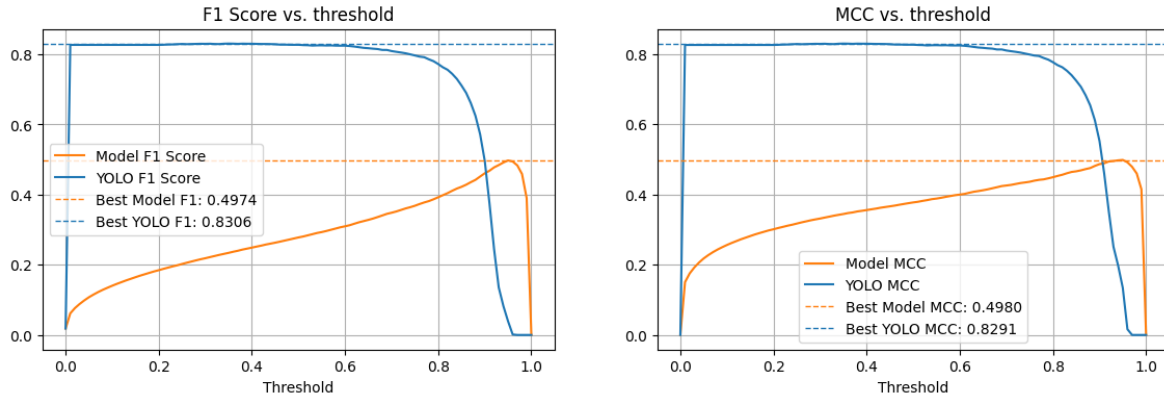


Figure 24. F1 Score and MCC of my model and YOLOv8 depending on the threshold.

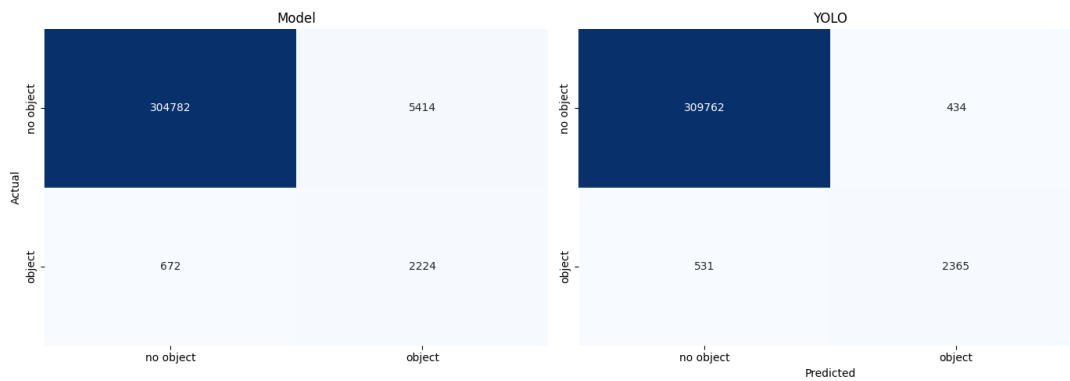


Figure 25. Confusion matrices of my model (threshold 0.85) and YOLOv8 (threshold 0.35) on the validation set.

The graphs show that my model is more than 1.5 times worse than YOLO in both F1 Score and MCC metrics. Confusion Matrices also clearly demonstrate that, although my model found almost as many actual cars (**True Positive**) as YOLO, it had more than ten times as many False Positives (**Type I Errors**), meaning it found cars where none exist or created multiple bounding boxes for the same car. Using the Non-Maximum Suppression (NMS) algorithm improves the situation, but not significantly, and it also degrades the True Positive metric.

Obviously, the reason is the significantly simpler model architecture, as well as the simplified NMS.

13 Conclusion

My main goal for this project was to **understand how object detection works** and **build a detection model from scratch**. And I succeeded: my model, despite being significantly simplified compared to the real YOLO, was able to **detect over 75% of cars** in the validation set images. I think that's not bad for a first attempt. And while it's inferior to the real YOLO, I gained a wealth of new knowledge and experience during the project, which I'm sure I'll be able to put to good use again.

A significant portion of my model's false positive errors were due to neighboring grid cells believing they were the center of an object's bounding box and competing for it. Perhaps a more advanced implementation of the Non-Maximum Suppression (NMS) algorithm would have been better able to handle such duplicates.

Among the possible **ways of improvement**, I primarily see **multi-class object detection**, meaning bringing back the remaining classes I excluded for the sake of model simplicity. I also didn't use all the existing model features, such as alpha — the angle at which the car faces the camera. This is crucial for real-world autonomous driving, because if the camera sees a car in front of it and it's moving in the same direction, it's not the same as if it's moving toward you.

When it comes to autonomous driving specifically, simply analyzing camera images and detecting objects is weak. All objects are moving, some obscure others from the camera, and instead of individual images of objects, the autonomous driving system must construct a 3D representation of the surrounding space, including what objects are present, where they are moving, and so on. Each new frame shouldn't be isolated from the previous ones, but rather complement the previous ones, clarifying the position of objects and improving the prediction of their motion. But this is a completely different level of deep learning, and I don't yet understand how to implement it. However, I am very interested in this area; perhaps this task will form the basis of my next project.

Thank you very much for reviewing my assignment!

References

- [1] A. Geiger, P. Lenz, R. Urtasun. Are we ready for autonomous driving? The KITTI vision benchmark suite. *CVPR*, 2012. <https://www.cvlibs.net/datasets/kitti/>
- [2] J. Redmon, S. Divvala, R. Girshick, A. Farhadi. You Only Look Once: Unified, Real-Time Object Detection. *CVPR*, 2016.
- [3] J. Redmon, A. Farhadi. YOLO9000: Better, Faster, Stronger. *CVPR*, 2017.
- [4] G. Jocher, A. Chaurasia, J. Qiu. Ultralytics YOLOv8, 2023. <https://github.com/ultralytics/ultralytics>
- [5] L. Wood, Z. Tan, I. Stenbit, J. Bischof, S. Zhu, F. Chollet et al. KerasCV, 2022. <https://github.com/keras-team/keras-cv>
- [6] T.-Y. Lin, M. Maire, S. Belongie et al. Microsoft COCO: Common Objects in Context. *ECCV*, 2014.
- [7] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, D. Ren. Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression. *AAAI*, 2020.
- [8] S. Ioffe, C. Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *ICML*, 2015.
- [9] J. Bergstra, Y. Bengio. Random Search for Hyper-Parameter Optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.
- [10] D. Chicco, G. Jurman. The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21:6, 2020.

A Supplementary code

This appendix contains the remaining code of the notebook: imports, helper functions and the code that produced the figures. The complete notebook is available in the project repository.

Listing 14. Imports and dataset file lists.

```

1  import os
2  os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
3
4  import glob
5  import math
6  import time
7  import random
8  import numpy as np
9  import pandas as pd
10 import seaborn as sns
11 import matplotlib.pyplot as plt
12 from PIL import Image, ImageDraw
13
14 import cv2
15 import keras_cv
16 import tensorflow as tf
17 from tensorflow import keras
18 from tensorflow.keras import layers, models
19 from sklearn.model_selection import train_test_split
20 from sklearn.metrics import confusion_matrix, roc_curve, auc, classification_report,
    precision_score, recall_score, f1_score, matthews_corrcoef
21
22 image_files = sorted(glob.glob('training/image_2/*.png'))
23 label_files = sorted(glob.glob('training/label_2/*.txt'))
24
25 print(len(image_files), len(label_files))
26
27 image = Image.open(image_files[0])
28 print(image.mode, image.size)

```

Listing 15. Plotting code for Figure 2.

```

1  fig, ax = plt.subplots(1, 2, figsize=(12, 4))
2
3  # Left plot: objects per image
4  df.groupby('image_id')['type'].count().value_counts().sort_index().plot(kind='bar', ax=ax[0])
5  ax[0].set_xlabel('Number of objects per image')
6  ax[0].set_ylabel('Number of images')
7  ax[0].set_title('Distribution of object counts per image')
8  ax[0].grid(True)
9
10 # Right plot: class distribution
11 df['type'].value_counts().plot(kind='bar', ax=ax[1])
12 ax[1].set_xlabel('')
13 ax[1].set_ylabel('Number of objects')
14 ax[1].set_title('Object distribution by class')
15 ax[1].tick_params(axis='x', rotation=45)
16 ax[1].grid(True)

```

```

17
18 plt.tight_layout()
19 plt.show()
20
21 print('Mean of objects per image:', df.groupby('image_id')['type'].count().mean())

```

Listing 16. Plotting code for Figure 3.

```

1 fig, ax = plt.subplots(1, 2, figsize=(12, 4))
2
3 df.boxplot(column='width', by='type', ax=ax[0])
4 ax[0].tick_params(axis='x', rotation=45)
5 ax[0].set_title('Bounding box width by class')
6 ax[0].set_xlabel('')
7 ax[0].set_ylabel('Width (pixels)')
8
9 df.boxplot(column='height', by='type', ax=ax[1])
10 ax[1].tick_params(axis='x', rotation=45)
11 ax[1].set_title('Bounding box height by class')
12 ax[1].set_xlabel('')
13 ax[1].set_ylabel('Height (pixels)')
14
15 plt.suptitle('')
16 plt.tight_layout()
17 plt.show()

```

Listing 17. Plotting code for Figure 4.

```

1 # scatter width vs height
2 plt.figure(figsize=(12, 5))
3 for i, t in enumerate(df['type'].unique()):
4     mask = df['type'] == t
5     plt.scatter(df['width'][mask], df['height'][mask], s=5, alpha=0.3, label=t)
6 plt.xlabel('Width (pixels)')
7 plt.ylabel('Height (pixels)')
8 plt.title('Bounding box width vs height')
9 plt.legend()
10
11 plt.tight_layout()
12 plt.show()

```

Listing 18. Plotting code for Figure 5.

```

1 fig, ax = plt.subplots(2, 2, figsize=(15, 8))
2
3 ax[0,0].hist(df['width'], bins=40)
4 ax[0,0].set_title('Bounding box width')
5 ax[0,0].set_xlabel('Pixels')
6 ax[0,0].set_ylabel('Number of bounding boxes')
7 ax[0,0].grid(True)
8
9 ax[0,1].hist(df['height'], bins=40)
10 ax[0,1].set_title('Bounding box height')
11 ax[0,1].set_ylabel('Number of bounding boxes')

```

```

12 ax[0,1].set_xlabel('Pixels')
13 ax[0,1].grid(True)
14
15 ax[1,0].hist(df['area'], bins=40)
16 ax[1,0].set_title('Bounding box area')
17 ax[1,0].set_ylabel('Number of bounding boxes')
18 ax[1,0].set_xlabel('Pixels ^2')
19 ax[1,0].grid(True)
20
21 ax[1,1].hist(df['aspect_ratio'], bins=40)
22 ax[1,1].set_title('Aspect ratio distribution')
23 ax[1,1].set_ylabel('Number of bounding boxes')
24 ax[1,1].set_xlabel('Aspect ratio (width / height)')
25 ax[1,1].grid(True)
26
27 plt.tight_layout()
28 plt.show()

```

Listing 19. Plotting code for Figure 6.

```

1 fig, ax = plt.subplots(1, 2, figsize=(12, 4))
2
3 ax[0].scatter(df_cars['width'], df_cars['height'], s=5, alpha=0.3)
4 ax[0].set_xlabel('Width (pixels)')
5 ax[0].set_ylabel('Height (pixels)')
6 ax[0].set_title('Bounding box width vs height')
7
8 ax[1].hist(df_cars['aspect_ratio'], bins=50)
9 ax[1].set_title('Aspect ratio distribution for Car')
10 ax[1].set_xlabel('Aspect ratio (width / height)')
11 ax[1].set_ylabel('Number of bounding boxes')
12
13 plt.tight_layout()
14 plt.show()

```

Listing 20. Plotting code for Figure 7.

```

1 groups = ['far', 'mid', 'near']
2 fig, axes = plt.subplots(1, 3, figsize=(15,4))
3
4 for ax, group in zip(axes, groups):
5     d = df_cars[df_cars['distance'] == group]
6     heat, x, y = np.histogram2d(d['xmin']+d['width']/2, d['ymin']+d['height']/2,
7                                bins=[40, 40], range=[[0, 1224], [0, 370]])
8     ax.imshow(heat.T, origin='lower', extent=[0, 1224, 0, 370])
9     ax.set_title(f'{group} distance')
10    ax.axis('off')
11
12 plt.tight_layout()
13 plt.show()

```

Listing 21. Plotting code for Figure 8.

```

1 # I cannot use the usual plt.subplots() here
2 # because the polar plot is not compatible with this layout.
3
4 fig = plt.figure(figsize=(15,5))
5 gs = fig.add_gridspec(1, 3)
6
7 position = (df_cars['xmin'] + df_cars['width'] / 2).apply(lambda x: 'left' if x < 612 else 'right')
8 ax0 = fig.add_subplot(gs[0,0])
9 position.value_counts().plot(kind='bar', ax=ax0)
10 ax0.tick_params(axis='x', rotation=45)
11 ax0.set_ylabel('Number of cars')
12 ax0.set_title('Left / Right distribution')
13 ax0.set_xlabel('')
14
15 ax1 = fig.add_subplot(gs[0,1])
16 df_cars.boxplot(column='aspect_ratio', by='distance', ax=ax1)
17 ax1.set_title('Aspect ratio by distance')
18 ax1.set_ylabel('Aspect ratio')
19 ax1.set_xlabel('')
20
21 ax2 = fig.add_subplot(gs[0,2], polar=True)
22 ax2.hist(df_cars['alpha'].values, bins=36, alpha=0.7) # polar histogram
23 ax2.set_title('Orientation distribution (alpha)')
24
25 plt.suptitle('')
26 plt.tight_layout()
27 plt.show()

```

Listing 22. Drawing ground-truth bounding boxes, Figure 9.

```

1 np.random.seed(19)
2 for id in np.random.choice(df_cars['image_id'].unique(), size=3, replace=False):
3     img = cv2.imread(f"training/image_2/{id}.png")
4     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
5     for _, row in df_cars[df_cars['image_id'] == id].iterrows():
6         cv2.rectangle(img, (int(row.xmin), int(row.ymin)), (int(row.xmax), int(row.ymax)), (255,
7             0, 0), 2)
8     plt.figure(figsize=(10, 4))
9     plt.imshow(img)
10    plt.axis("off")
11    plt.show()

```

Listing 23. Image loading, grid rendering and decoding check, Figure 10.

```

1 def load_image_resize(id):
2     img = cv2.imread(f'training/image_2/{id}.png')
3     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
4     img = cv2.resize(img, (1248, 384))
5     return img
6
7 def image_grid(img):
8     for col in range(40):
9         x = int(col * 1248 / 39)
10        cv2.line(img, (x, 0), (x, 384), (0, 255, 0), 1)
11    for row in range(13):

```

```

12     y = int(row * 384 / 12)
13     cv2.line(img, (0, y), (1248, y), (0, 255, 0), 1)
14     return img
15
16 img = image_grid(load_image_resize('000047'))
17
18 row = df_cars[df_cars['image_id']=='000047'].iloc[2]
19 i, j = row['cell_i'], row['cell_j']
20 cx = (j + row['bx']) * 32
21 cy = (i + row['by']) * 32
22 hw = np.exp(row['bw']) * 32 / 2
23 hh = np.exp(row['bh']) * 32 / 2
24
25 cv2.rectangle(img, (int(cx-hw), int(cy-hh)), (int(cx+hw), int(cy+hh)), (255, 0, 0), 2)
26
27 cv2.rectangle(img, (j*32, i*32), ((j+1)*32, (i+1)*32), (255, 255, 255), 3)
28
29 plt.figure(figsize=(10, 4))
30 plt.imshow(img)
31 plt.axis('off')
32 plt.show()

```

Listing 24. Plotting code for Figure 11.

```

1 plt.figure(figsize=(8, 4))
2 for i, r in enumerate(results):
3     plt.plot(r['history']['val_loss'], label=f'trial #{i}')
4 plt.xlabel('Epoch')
5 plt.ylabel('Loss')
6 plt.title('Validation Loss per trial')
7 plt.legend()
8 plt.grid(True)
9 plt.show()

```

Listing 25. Full training and evaluation cell, including the plots of Figure 12.

```

1 best_trial = min(results, key=lambda r: r['best_val_loss'])
2
3 fig, ax = plt.subplots(1, 2, figsize=(14,4))
4
5 ax[0].plot(results[0]['history']['loss'], label='Training Loss')
6 ax[0].plot(results[0]['history']['val_loss'], label='Validation Loss')
7 ax[0].set_xlabel('Epoch')
8 ax[0].set_ylabel('Loss')
9 ax[0].set_ylim(0,5)
10 ax[0].set_xlim(0, best_trial['best_epoch']+2)
11 ax[0].set_title('Training loss / Validation loss, trial #0')
12 ax[0].legend()
13 ax[0].grid(True)
14
15
16 random.seed(42)
17 np.random.seed(42)
18 tf.random.set_seed(42)
19

```

```

20 model = build_model()
21 model.compile(optimizer=tf.keras.optimizers.Adam(best_trial['config']['lr']),
22               loss=make_loss(best_trial['config']['lambda_coord'],
23                               best_trial['config']['lambda_noobj']))
24
25 history = model.fit(
26     data_generator(train, BATCH_SIZE),
27     epochs=best_trial['best_epoch'],
28     steps_per_epoch=len(train) // BATCH_SIZE,
29     validation_data=data_generator(val, BATCH_SIZE, shuffle=False),
30     validation_steps=max(1, len(val) // BATCH_SIZE),
31     verbose=0
32 )
33
34 pred, target = [], []
35 for id in val:
36     pred.append(model.predict(load_image_resize_norm(id)[None, ...], verbose=0)[0])
37     target.append(encode_target(id))
38
39 # convert lists into arrays
40 val_pred = np.stack(pred, axis=0)
41 val_target = np.stack(target, axis=0)
42
43 def sigmoid(z):
44     return 1.0/(1.0+np.exp(-z))
45
46 true_objects = val_target[..., 4].reshape(-1) # N * 12 * 39
47 pred_objects = sigmoid(val_pred[..., 4]).reshape(-1) # N * 12 * 39
48
49 fpr, tpr, thresholds = roc_curve(true_objects, pred_objects)
50 ax[1].plot(fpr, tpr, label=f'AUC = {auc(fpr, tpr):.2f}')
51 ax[1].plot([0, 1], [0, 1], 'k--') # baseline
52 ax[1].set_xlabel('False Positive Rate')
53 ax[1].set_ylabel('True Positive Rate')
54 ax[1].set_title('ROC Curve')
55 ax[1].legend(loc='lower right')
56
57 plt.tight_layout()
58 plt.show()

```

Listing 26. Objectness heatmap, Figure 13.

```

1 def obj_heatmap(matrix):
2     plt.figure(figsize=(16,5))
3     img = plt.imshow(matrix, cmap='Reds')
4
5     for i in range(12):
6         for j in range(39):
7             plt.text(j, i, f'{matrix[i, j]:.2f}',
8                       ha='center', va='center', color='black')
9
10    plt.axis('off')
11    plt.show()
12
13 obj_heatmap(objects)

```

Listing 27. Overlaying predicted and true cells on the image, Figure 14.

```

1  img_pred = image_grid(load_image_resize('000047'))
2
3  def image_obj_cell(img, objects, threshold=0.5):
4      for i in range(12):
5          for j in range(39):
6              if objects[i][j] > threshold:
7                  xmin, ymin = j * 32, i * 32
8                  cv2.rectangle(img, (xmin, ymin), (xmin+32, ymin+32), (255, 255, 255), -1)
9                  cv2.putText(img, f"{objects[i][j]:.2f}", (xmin + 2, ymin + 15),
10                             cv2.FONT_HERSHEY_SIMPLEX, 0.4, (255, 0, 0), 1)
11
12  image_obj_cell(img_pred, objects)
13
14  plt.figure(figsize=(16,5))
15  plt.imshow(img_pred)
16  plt.axis('off')
17  plt.title('Predicted objects')
18  plt.show()
19
20  img_true = image_grid(load_image_resize('000047'))
21  image_obj_cell(img_true, encode_target('000047')[..., 4])
22  plt.figure(figsize=(16,5))
23  plt.imshow(img_true)
24  plt.axis('off')
25  plt.title('True objects')
26  plt.show()

```

Listing 28. Metrics versus threshold, Figure 15.

```

1  thresholds = np.arange(0.0, 1.01, 0.01)
2
3  plt.figure(figsize=(10,5))
4
5  precisions, recalls, f1s, mccs = [], [], [], []
6  for thr in thresholds:
7      pred_obj = (pred_objects >= thr).astype(int)
8      precisions.append(precision_score(true_objects, pred_obj, zero_division=1))
9      recalls.append(recall_score(true_objects, pred_obj))
10     f1s.append(f1_score(true_objects, pred_obj))
11     mccs.append(matthews_corrcoef(true_objects, pred_obj))
12
13  best_f1_idx = np.argmax(f1s)
14  best_f1_threshold = thresholds[best_f1_idx]
15  best_mcc_idx = np.argmax(mccs)
16  best_mcc_threshold = thresholds[best_mcc_idx]
17
18  plt.plot(thresholds, precisions, label='Precision')
19  plt.plot(thresholds, recalls, label='Recall')
20  plt.plot(thresholds, f1s, label='F1 Score')
21  plt.plot(thresholds, mccs, label='MCC')
22  plt.axvline(0.5, color='gray', linestyle='--', label='Default threshold = 0.5')
23  plt.axvline(best_f1_threshold, color='green', linestyle='--',
24              label=f"Best F1: {f1s[best_f1_idx]:.4f}, threshold: {best_f1_threshold:.2f}")
25  plt.axvline(best_mcc_threshold, color='red', linestyle='--',

```

```

26         label=f"Best MCC: {mccs[best_mcc_idx]:.4f}, threshold: {best_mcc_threshold:.2f}")
27 plt.xlabel('Threshold')
28 plt.ylabel('Metric Value')
29 plt.title('Metrics vs. threshold')
30 plt.legend()
31 plt.grid(True)
32 plt.show()

```

Listing 29. Confusion matrices, Figure 16.

```

1  fig, ax = plt.subplots(1, 3, figsize=(16, 5))
2
3  cm8 = confusion_matrix(true_objects, (pred_objects > 0.8).astype(int))
4  cm85 = confusion_matrix(true_objects, (pred_objects > 0.85).astype(int))
5  cm95 = confusion_matrix(true_objects, (pred_objects > 0.95).astype(int))
6
7  labels = ['no object', 'object']
8
9  sns.heatmap(cm8, annot=True, fmt='d', cmap='Blues', ax=ax[0], cbar=False,
10             xticklabels=labels, yticklabels=labels)
11 ax[0].set_xlabel('')
12 ax[0].set_ylabel('Actual')
13 ax[0].set_title('threshold 0.8')
14
15 sns.heatmap(cm85, annot=True, fmt='d', cmap='Blues', ax=ax[1], cbar=False,
16            xticklabels=labels, yticklabels=labels)
17 ax[1].set_xlabel('Predicted')
18 ax[1].set_ylabel('')
19 ax[1].set_title('threshold 0.85')
20
21 sns.heatmap(cm95, annot=True, fmt='d', cmap='Blues', ax=ax[2],
22            xticklabels=labels, yticklabels=labels)
23 ax[2].set_xlabel('')
24 ax[2].set_ylabel('')
25 ax[2].set_title('threshold 0.95')
26
27 plt.tight_layout()
28 plt.show()

```

Listing 30. Predictions with NMS on random validation images, Figure 19.

```

1  np.random.seed(42)
2
3  for i in np.random.choice(range(len(val)), size=5, replace=False):
4      img = load_image_resize(val[i])
5
6      boxes, scores, _, _ = decode_prediction(val_pred[i], threshold=0.85)
7      for j in nms(boxes, scores, iou_thr=0.2):
8          cv2.rectangle(img, (int(boxes[j][0]), int(boxes[j][1])), (int(boxes[j][2]),
9                                int(boxes[j][3])), (255, 0, 0), 2)
10
11     plt.figure(figsize=(16,5))
12     plt.imshow(img)
13     plt.axis('off')
14     plt.show()

```

Listing 31. Full YOLO evaluation cell, including the plots of Figure 20.

```

1  fig, ax = plt.subplots(1, 2, figsize=(14,4))
2
3  ax[0].plot(yolo_history.history['loss'], label='Training Loss')
4  ax[0].plot(yolo_history.history['val_loss'], label='Validation Loss')
5  ax[0].set_xlabel('Epoch')
6  ax[0].set_ylabel('Loss')
7  ax[0].set_title('Training loss / Validation loss, trial #0')
8  ax[0].legend()
9  ax[0].grid(True)
10
11
12 def yolo_decode(pred):
13     result = []
14     for n, confidence in enumerate(pred['confidence']):
15
16         cells = np.zeros((12, 39), dtype=np.float32)
17
18         for m, conf in enumerate(confidence):
19             if conf < 0: continue
20
21             box = pred['boxes'][n][m]
22             j = ((box[0] + box[2]) / 2 / 32).astype(int).clip(0, 38)
23             i = ((box[1] + box[3]) / 2 / 32).astype(int).clip(0, 11)
24             if cells[i, j] > conf: continue
25             cells[i, j] = conf
26
27         result.append({
28             'boxes': pred['boxes'][n],
29             'cells': cells,
30             'confidence': pred['confidence'][n]
31         })
32     return result
33
34 yolo_pred = []
35 for id in val:
36     yolo_pred.extend(yolo_decode(yolo.predict(load_image_resize_norm(id)[None, ...], verbose=0)))
37
38
39 yolo_objects = np.stack([pred['cells'] for pred in yolo_pred], axis=0).reshape(-1)
40
41 fpr, tpr, thresholds = roc_curve(true_objects, yolo_objects)
42 ax[1].plot(fpr, tpr, label=f'AUC = {auc(fpr, tpr):.2f}')
43 ax[1].plot([0, 1], [0, 1], 'k--') # baseline
44 ax[1].set_xlabel('False Positive Rate')
45 ax[1].set_ylabel('True Positive Rate')
46 ax[1].set_title('ROC Curve')
47 ax[1].legend(loc='lower right')
48
49 plt.tight_layout()
50 plt.show()

```

Listing 32. YOLO metrics versus threshold, Figure 21.

```

1 thresholds = np.arange(0.0, 1.01, 0.01)
2
3 plt.figure(figsize=(10,5))
4
5 precisions, recalls, f1s, mccs = [], [], [], []
6 for thr in thresholds:
7     pred_obj = (yolo_objects >= thr).astype(int)
8     precisions.append(precision_score(true_objects, pred_obj, zero_division=1))
9     recalls.append(recall_score(true_objects, pred_obj))
10    f1s.append(f1_score(true_objects, pred_obj))
11    mccs.append(matthews_corrcoef(true_objects, pred_obj))
12
13 best_f1_idx = np.argmax(f1s)
14 best_f1_threshold = thresholds[best_f1_idx]
15 best_mcc_idx = np.argmax(mccs)
16 best_mcc_threshold = thresholds[best_mcc_idx]
17
18 plt.plot(thresholds, precisions, label='Precision')
19 plt.plot(thresholds, recalls, label='Recall')
20 plt.plot(thresholds, f1s, label='F1 Score')
21 plt.plot(thresholds, mccs, label='MCC')
22 plt.axvline(0.5, color='gray', linestyle='--', label='Default threshold = 0.5')
23 plt.axvline(best_f1_threshold, color='green', linestyle='--',
24             label=f"Best F1: {f1s[best_f1_idx]:.4f}, threshold: {best_f1_threshold:.2f}")
25 plt.axvline(best_mcc_threshold, color='red', linestyle='--',
26             label=f"Best MCC: {mccs[best_mcc_idx]:.4f}, threshold: {best_mcc_threshold:.2f}")
27 plt.xlabel('Threshold')
28 plt.ylabel('Metric Value')
29 plt.title('Metrics vs. threshold')
30 plt.legend()
31 plt.grid(True)
32 plt.show()

```

Listing 33. YOLO predicted cells, Figure 22.

```

1 pred_single_image_yolo = yolo_decode(yolo.predict(load_image_resize_norm('000047')[None, ...],
2                                         verbose=0))[0]
3
4 img_yolo = image_grid(load_image_resize('000047'))
5
6 image_obj_cell(img_yolo, pred_single_image_yolo['cells'], threshold=0.35) # use the best threshold
7     value
8
9 plt.figure(figsize=(16,5))
10 plt.imshow(img_yolo)
11 plt.axis('off')
12 plt.title('Yolo objects')
13 plt.show()
14
15 img_true = image_grid(load_image_resize('000047'))
16 image_obj_cell(img_true, encode_target('000047')[..., 4])
17
18 plt.figure(figsize=(16,5))
19 plt.imshow(img_true)
20 plt.axis('off')
21 plt.title('True objects')
22 plt.show()

```

Listing 34. YOLO bounding boxes, Figure 23.

```

1  img = load_image_resize('000047')
2
3  for i, conf in enumerate(pred_single_image_yolo['confidence']):
4      if conf < 0.35: continue
5      b = pred_single_image_yolo['boxes'][i]
6      cv2.rectangle(img, (int(b[0]), int(b[1])), (int(b[2]), int(b[3])), (255, 0, 0), 2)
7
8  plt.figure(figsize=(16,5))
9  plt.imshow(img)
10 plt.axis('off')
11 plt.show()

```

Listing 35. Model comparison plots, Figure 24.

```

1  thresholds = np.arange(0.0, 1.01, 0.01)
2
3  fig, ax = plt.subplots(1, 2, figsize=(14,4))
4
5  f1s, mccs, yolo_f1s, yolo_mccs = [], [], [], []
6  for thr in thresholds:
7      pred_obj = (pred_objects >= thr).astype(int)
8      f1s.append(f1_score(true_objects, pred_obj))
9      mccs.append(matthews_corrcoef(true_objects, pred_obj))
10
11     yolo_obj = (yolo_objects >= thr).astype(int)
12     yolo_f1s.append(f1_score(true_objects, yolo_obj))
13     yolo_mccs.append(matthews_corrcoef(true_objects, yolo_obj))
14
15 best_f1_idx = np.argmax(f1s)
16 best_f1_threshold = thresholds[best_f1_idx]
17 best_mcc_idx = np.argmax(mccs)
18 best_mcc_threshold = thresholds[best_mcc_idx]
19
20 yolo_best_f1_idx = np.argmax(yolo_f1s)
21 yolo_best_mcc_idx = np.argmax(yolo_mccs)
22
23 ax[0].plot(thresholds, f1s, color='#ff7f0e', label='Model F1 Score')
24 ax[0].plot(thresholds, yolo_f1s, color='#1f77b4', label='YOLO F1 Score')
25 ax[0].axhline(f1s[best_f1_idx], color='#ff7f0e', linestyle='--', lw=1,
26              label=f"Best Model F1: {f1s[best_f1_idx]:.4f}")
27 ax[0].axhline(yolo_f1s[yolo_best_f1_idx], color='#1f77b4', linestyle='--', lw=1,
28              label=f"Best YOLO F1: {yolo_f1s[yolo_best_f1_idx]:.4f}")
29 ax[0].set_xlabel('Threshold')
30 ax[0].set_title('F1 Score vs. threshold')
31 ax[0].grid(True)
32 ax[0].legend()
33
34 ax[1].plot(thresholds, mccs, color='#ff7f0e', label='Model MCC')
35 ax[1].plot(thresholds, yolo_mccs, color='#1f77b4', label='YOLO MCC')
36 ax[1].axhline(mccs[best_mcc_idx], color='#ff7f0e', linestyle='--', lw=1,
37              label=f"Best Model MCC: {mccs[best_mcc_idx]:.4f}")
38 ax[1].axhline(yolo_mccs[yolo_best_mcc_idx], color='#1f77b4', linestyle='--', lw=1,
39              label=f"Best YOLO MCC: {yolo_mccs[yolo_best_mcc_idx]:.4f}")
40 ax[1].set_xlabel('Threshold')

```

```
41 ax[1].set_title('MCC vs. threshold')
42 ax[1].grid(True)
43 ax[1].legend()
44
45 plt.show()
```

Listing 36. Confusion matrices of both models, Figure 25.

```
1 fig, ax = plt.subplots(1, 2, figsize=(14, 5))
2
3 yolo_cm = confusion_matrix(true_objects, (yolo_objects > 0.35).astype(int))
4
5 labels = ['no object', 'object']
6
7 sns.heatmap(cm85, annot=True, fmt='d', cmap='Blues', ax=ax[0], cbar=False,
8             xticklabels=labels, yticklabels=labels)
9 ax[0].set_xlabel('')
10 ax[0].set_ylabel('Actual')
11 ax[0].set_title('Model')
12
13 sns.heatmap(yolo_cm, annot=True, fmt='d', cmap='Blues', ax=ax[1], cbar=False,
14             xticklabels=labels, yticklabels=labels)
15 ax[1].set_xlabel('Predicted')
16 ax[1].set_ylabel('')
17 ax[1].set_title('YOLO')
18
19 plt.tight_layout()
20 plt.show()
```